

Commitment Schemes

DSC 4110/6111: Modern Cryptography and AI/ML Security

Dr. Laltu Sardar

School of Data Science,
Indian Institute of Science Education and Research Thiruvananthapuram (IISER TVM)



September 29, 2026

Table of Content

1 Commitment Schemes

Commitment Schemes

Commitment Scheme: The Basic Idea

A commitment scheme allows a party to commit to a value now, without revealing the value. Later, the party can reveal the value and prove that it was the same value that was committed earlier.

Two Main Properties

- **Hiding:** The commitment does not reveal the committed value.
- **Binding:** After committing, the sender cannot change the committed value later.

The basic interaction is:

$$\text{Commit: } m, r \longrightarrow C$$

$$\text{Open: } (m, r) \longrightarrow C$$

where m is the committed value and r is random.

Security of a Commitment Scheme

A secure commitment scheme should provide two fundamental properties.

Hiding

Given only the commitment C , the receiver should not be able to learn the committed value m .

$$C \not\Rightarrow m$$

Binding

After producing a commitment C , the sender should not be able to open it as two different values.

$$(m, r) \neq (m', r') \implies \text{not both can open the same } C.$$

Hiding: cannot learn

Binding: cannot change

Example: Teacher Commits to a Future Grade

Suppose a teacher wants to commit today to whether a student's final grade will be at least B . The teacher does *not* want to reveal the answer now.

Define:

$$b = \begin{cases} 1, & \text{if final grade} \geq B, \\ 0, & \text{otherwise.} \end{cases}$$

The teacher commits to b now and reveals it only after the final result is available.

Desired Properties

Today:

Teacher commits to b without revealing b

Later:

Teacher reveals b and proves consistency

The teacher should not be able to change b after committing.

Hash-Based Commitment

Let H be a cryptographic hash function.

The teacher chooses a random value:

$$x \leftarrow \{0, 1\}^\lambda.$$

The commitment is:

$$c = H(b\|x)$$

where $b \in \{0, 1\}$ is the teacher's decision bit.

The teacher sends only:

c

to the student.

What does the student know?

The student sees c , but does not know b or x .

Therefore, the commitment hides the teacher's decision.

Opening the Commitment

After the final result is available, the teacher reveals:

$$(b, x).$$

The student computes:

$$H(b\|x)$$

and checks:

$$\boxed{H(b\|x) \stackrel{?}{=} c}$$

If the equality holds, the student accepts the opening.

Why can the teacher not change the decision?

Suppose the teacher initially committed to b but later tries to open the commitment as $b' \neq b$.

The teacher would need to find x' such that:

$$H(b\|x) = H(b'\|x').$$

Finding such an alternative opening should be computationally infeasible for a suitable cryptographic hash function.

Can the Teacher Commit to the Marks Instead?

Yes.

Suppose the student's final mark is:

$$m \in [0, 100].$$

The teacher can commit directly to m .

Choose:

$$x \leftarrow \{0, 1\}^\lambda$$

and compute:

$$c = H(m \| x)$$

The teacher sends only c to the student.

Later, after the final result, the teacher reveals:

$$(m, x).$$

The student verifies:

$$H(m \| x) \stackrel{?}{=} c$$

Thus, the teacher is committed to the same mark throughout.

Commitment vs. Zero-Knowledge Proof

There is an important difference between a commitment scheme and a zero-knowledge proof.

Commitment Scheme

The committed value is eventually revealed.

$$c = H(m||x) \longrightarrow \text{later reveal } (m, x)$$

The receiver can then verify the commitment.

Zero-Knowledge Proof

The prover wants to prove a statement about the hidden value *without revealing the value itself*.

Prove a property of m without revealing m

The New Challenge

Suppose the student committed to the final mark:

$$c = H(m||x).$$

Now suppose the student wants to prove:

$$m \geq 50$$

without revealing m .

Simply opening the commitment reveals:

$$m.$$

But we want:

Prove $m \geq 50$ without revealing m .

Similarly, we may want to prove:

$$m \in [50, 100]$$

or

$$m \in \{50, 60, 70, 80, 90, 100\}.$$

From Commitments to Zero-Knowledge Proofs

A commitment gives us a way to keep a value hidden while allowing us to refer to that value later.
For example:

$$C = \text{Com}(m; r).$$

Now we want to prove a statement about the hidden m :

$$\exists m, r : C = \text{Com}(m; r) \quad \wedge \quad m \in [50, 100].$$

The verifier should learn only that such an m exists.

$\text{Commit to } m \quad + \quad \text{Prove a property of } m$

This is the basic idea behind many modern zero-knowledge protocols.

Next: How can we prove such properties without opening the commitment?

Commitment Schemes Categories

Categories of Commitment Schemes

Commitment schemes can be classified in two orthogonal ways:

1 Based on how the commitment is constructed

- Hash-based commitments
- Discrete-logarithm-based commitments
- RSA-based commitments
- Lattice-based commitments
- Code-based commitments

2 Based on what is being committed

- Bit commitment
- Message/string commitment
- Integer/numerical commitment
- Vector commitment
- Polynomial commitment
- Set commitment

How we commit + What we commit to

Category I: Based on How We Commit

The commitment mechanism can be classified according to the underlying cryptographic primitive or hardness assumption.

- **Hash-based:**

$$C = H(m \parallel r)$$

- **Discrete-logarithm-based:**

$$C = g^m h^r$$

- **RSA-based:**

$$C = \text{RSA-based construction}$$

- **Lattice-based:**

$$C = Ar + Bm \pmod{q}$$

- **Code-based:**

$$C = \text{code-based construction}$$

The underlying primitive determines the security assumptions and computational properties of the commitment.

Category II: Based on What We Commit

The committed value can itself have different mathematical structures.

- **Bit:**

$$b \in \{0, 1\}$$

- **Message/String:**

$$m \in \{0, 1\}^n$$

- **Integer:**

$$x \in \mathbb{Z} \quad \text{or} \quad x \in [0, N]$$

- **Vector:**

$$\mathbf{x} = (x_1, \dots, x_n)$$

- **Polynomial:**

$$f(X) = \sum_{i=0}^d a_i X^i$$

- **Set:**

$$S = \{x_1, \dots, x_n\}$$

The Two Dimensions Are Independent

The two classifications describe different aspects of a commitment.

How is the commitment constructed?



Hash, Pedersen, RSA, Lattice, ...

What is being committed?



Bit, Integer, Vector, Polynomial, ...

For example, a **Pedersen commitment** specifies the construction:

$$C = g^m h^r.$$

The value m could represent:

a bit | an integer | a field element | another algebraic object.

Thus, the same commitment mechanism can be used for different types of committed data.

Committing different data

Committing to Different Types of Data

A commitment need not be restricted to a particular type of data.

- Bit, number, message, vector, set, document, etc.
- We can encode the data as a binary string:

$$x \longrightarrow \text{Encode}(x) \in \{0, 1\}^*$$

- Then use a hash-based commitment:

$$C = H(\text{Encode}(x) \parallel r),$$

where r is a random nonce.

Thus, conceptually,

Any data $\xrightarrow{\text{Encode}}$ Binary string $\xrightarrow{H(\cdot \parallel r)}$ Commitment
--

The important issue is not only committing to the data, but proving properties of the hidden data.

Acknowledgment:

- ChatGPT (OpenAI) was used to identify the content for the slides, which was subsequently verified by me.



Dr. Laltu Sardar, Assistant Professor, IISER Thiruvananthapuram
laltu.sardar@iisertvm.ac.in, laltu.sardar.crypto@gmail.com
Course webpage: https://laltu-sardar.github.io/courses/26_crypto.html.